

```
#!/usr/bin/env python3
```

```
"""
```

```
audit_manifest.py - static triage of an AndroidManifest.xml (text/XML form).
```

```
python3 audit_manifest.py AndroidManifest.xml [--json]
```

Flags exported components with no permission guard, plus risky app flags and permissions. "Exported" is NOT "vulnerable": every HIGH/MEDIUM needs a read of the component's code (what extras/URIs it trusts, what it does with them). Resource references (@id/0x...) are not resolved; backup rules, FileProvider paths and network-security config must be reviewed from res/xml separately.

```
"""
```

```
import json
```

```
import sys
```

```
import xml.etree.ElementTree as ET
```

```
A = "{http://schemas.android.com/apk/res/android}"
```

```
ORDER = {"HIGH": 0, "MEDIUM": 1, "LOW": 2, "INFO": 3}
```

```
RISKY_PERMS = {
```

```
    "MANAGE_EXTERNAL_STORAGE": "MEDIUM", "REQUEST_INSTALL_PACKAGES": "MEDIUM",
```

```
    "SYSTEM_ALERT_WINDOW": "MEDIUM", "BIND_ACCESSIBILITY_SERVICE": "HIGH",
```

```
    "READ_SMS": "MEDIUM", "RECEIVE_SMS": "MEDIUM", "READ_CALL_LOG": "MEDIUM",
```

```
    "RECORD_AUDIO": "LOW", "READ_CONTACTS": "LOW", "ACCESS_FINE_LOCATION": "LOW",
```

```
    "QUERY_ALL_PACKAGES": "LOW", "WRITE_EXTERNAL_STORAGE": "LOW",
```

```
    "REQUEST_IGNORE_BATTERY_OPTIMIZATIONS": "INFO",
```

```
}
```

```
# protected system broadcasts: other apps cannot send these
```

```
SYSTEM_ACTIONS = {
```

```
    "android.intent.action.SCREEN_ON", "android.intent.action.SCREEN_OFF",
```

```
    "android.intent.action.BOOT_COMPLETED", "android.intent.action.MY_PACKAGE_REPLACED",
```

```
    "android.intent.action.TIME_SET", "android.intent.action.TIMEZONE_CHANGED",
```

```
}
```

```
def main(path, as_json):
```

```
    root = ET.parse(path).getroot()
```

```
    pkg = root.get("package", "?")
```

```
    sdk = root.find("uses-sdk")
```

```
    target = int(sdk.get(A + "targetSdkVersion", "0")) if sdk is not None else 0
```

```
    app = root.find("application")
```

```
    out = []
```

```
    def add(sev, where, msg):
```

```
        out.append((sev, where, msg))
```

```
    # --- app-level flags ---
```

```
    if target and target < 30:
```

```
        add("INFO", "uses-sdk", f"targetSdk={target}: legacy platform behaviour (storage, background limits). "
```

```
        "Sometimes deliberate (Termux-style exec from app data) but opts out of newer
```

```

protections.")
    if app is not None:
        if app.get(A + "debuggable") == "true":
            add("HIGH", "application", "debuggable=true")
        if app.get(A + "allowBackup", "true") == "true":
            add("MEDIUM", "application", "allowBackup=true: app data extractable via adb/cloud
backup; "
                "review backup rules (fullBackupContent / dataExtractionRules)")
        ct = app.get(A + "usesCleartextTraffic")
        if ct == "true" or (ct is None and 0 < target < 28):
            add("MEDIUM", "application", "cleartext HTTP permitted")
        if app.get(A + "requestLegacyExternalStorage") == "true":
            add("LOW", "application", "requestLegacyExternalStorage=true")

    for p in root.findall("uses-permission"):
        short = p.get(A + "name", "").rsplit(".", 1)[-1]
        if short in RISKY_PERMS:
            add(RISKY_PERMS[short], "uses-permission", short)

# --- components ---
for el in (app.iter() if app is not None else []):
    if el.tag not in ("activity", "activity-alias", "service", "receiver", "provider"):
        continue
    name = el.get(A + "name", "?")
    filters = el.findall("intent-filter")
    attr = el.get(A + "exported")
    exported = (attr == "true") if attr is not None else bool(filters)
    if not exported:
        continue
    actions = sorted({a.get(A + "name", "") for f in filters for a in f.findall("action")})
    perm = el.get(A + "permission")
    if el.tag == "provider":
        perm = perm or (el.get(A + "readPermission") and el.get(A + "writePermission"))
    if perm:
        continue # guarded; confirm the permission is signature-level if custom

    is_launcher = "android.intent.action.MAIN" in actions
    share_view = any(x.endswith(("SEND", "SEND_MULTIPLE", "VIEW")) for x in actions)
    sys_only = bool(actions) and all(a in SYSTEM_ACTIONS for a in actions)
    grant = el.get(A + "grantUriPermissions") == "true"

    if is_launcher:
        sev, msg = "INFO", "launcher entry (expected)"
    elif el.tag == "provider":
        sev, msg = "HIGH", "exported provider, no permission" + (" + grantUriPermissions" if
grant else "") + \
            ": any app can query; check for data leaks, path traversal, SQL injection"
    elif el.tag == "service":
        sev, msg = "HIGH", "exported service, no permission: any app can bind/start it"
    elif el.tag == "receiver" and sys_only:
        sev, msg = "LOW", "exported receiver for system-only actions: set exported=false"
    elif el.tag == "receiver":
        sev, msg = "MEDIUM", f"exported receiver, no permission, actions={actions}: any app
can send these"
    elif "tooling" in name or "PreviewActivity" in name:

```

```

        sev, msg = "MEDIUM", "debug tooling shipped in release build (use
debugImplementation)"
    elif share_view:
        sev, msg = "LOW", "share/view target (expected); treat all incoming data as untrusted"
    else:
        sev, msg = "MEDIUM", "exported activity with no intent-filter: any app can launch it
with " \
        "crafted extras; check what it executes/loads"
    add(sev, f"{el.tag} {name}", msg)

out.sort(key=lambda r: (ORDER[r[0]], r[1]))
if as_json:
    print(json.dumps([dict(severity=s, where=w, message=m) for s, w, m in out], indent=2))
else:
    print(f"{pkg} targetSdk={target}")
    for s, w, m in out:
        print(f"[{s:6}] {w}\n          {m}")
    counts = {k: sum(1 for r in out if r[0] == k) for k in ORDER}
    print("\n" + " ".join(f"{k}={v}" for k, v in counts.items()))

if __name__ == "__main__":
    args = [a for a in sys.argv[1:] if not a.startswith("--")]
    if not args:
        sys.exit(__doc__)
    main(args[0], "--json" in sys.argv)

```